

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **3**
Topic : **4**

Redis - Key-Value Database

Objectives

This session will give the knowledge about

- Redis introduction and basics
- Redis installation on Windows

What is Redis?

Redis is an **open source (BSD licensed)**, **in-memory data structure store**, **used as a database, cache and message broker**.

It supports data structures such as strings, hashes, lists, sets, sorted sets with range queries, bitmaps, hyperloglogs, geospatial indexes with radius queries and streams.

Redis has built-in replication, Lua scripting, LRU eviction, transactions and different levels of on-disk persistence, and provides high availability via Redis Sentinel and automatic partitioning with Redis Cluster.

Redis – The Background

Redis is written in ANSI C and works in most POSIX systems like Linux, *BSD, OS X without external dependencies.

Linux and OS X are the two operating systems where Redis is developed and tested the most, and we recommend using Linux for deploying.

Redis may work in Solaris-derived systems like SmartOS, but the support is best effort.

There is no official support for Windows builds.

How Redis works?

In order to achieve its outstanding performance, Redis works with an in-memory dataset.

Depending on your use case:

- you can persist it either by dumping the dataset to disk every once in a while, or by appending each command to a log.
- Persistence can be optionally disabled, if you just need a feature-rich, networked, in-memory cache.

Redis – In memory DB

When we say in-memory key-value store, by that we mean that the key-value pairs are stored in primary memory(RAM).

So we can say that **Redis stored data in RAM in form of key-value pairs**. In **Redis**, **key** has to be a string but **value** can be a string, list, set, sorted set or hash.

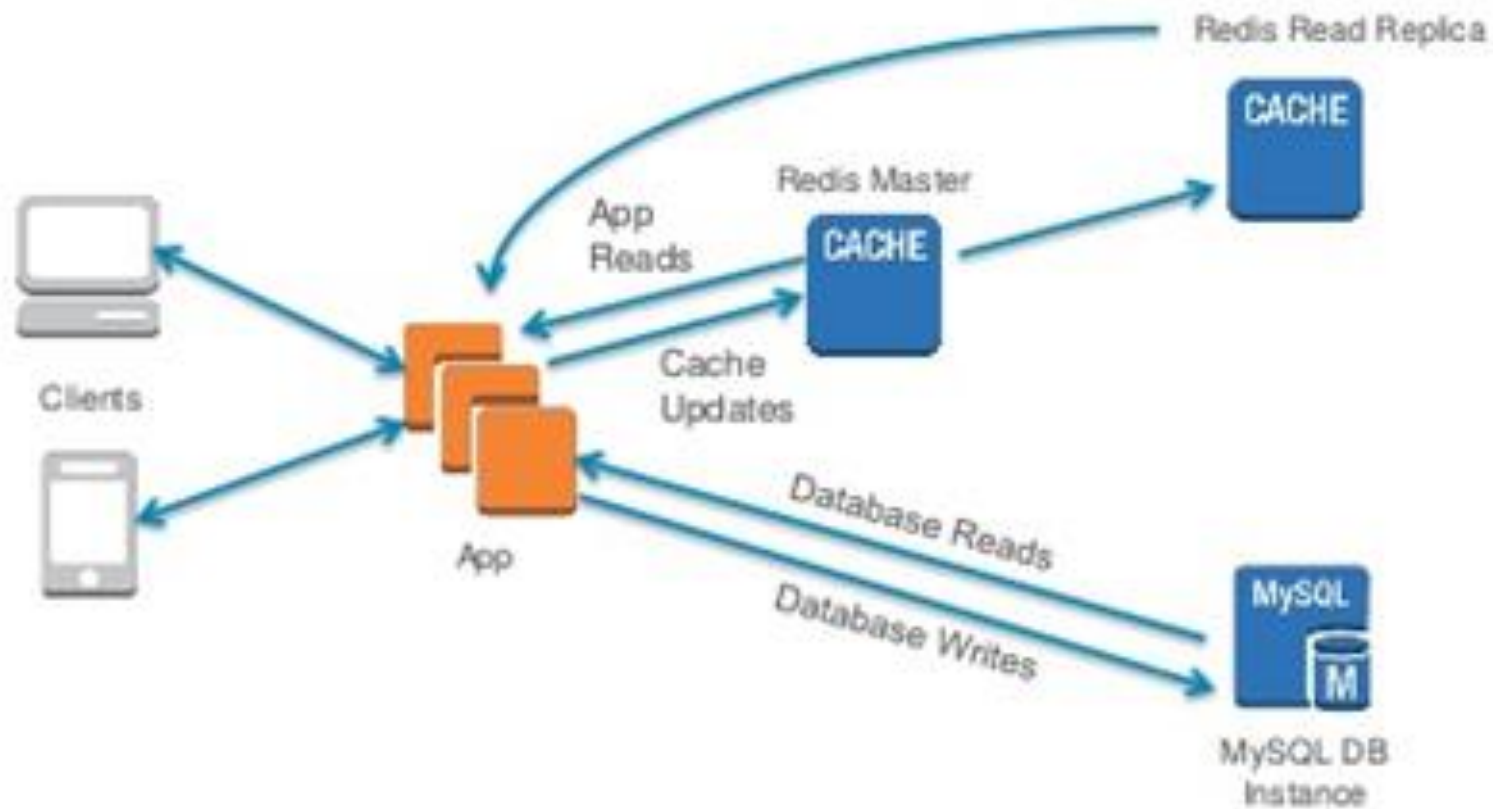
These are some example of Redis key-values pairs

name="narayan"

profession=["web", "mobile"]

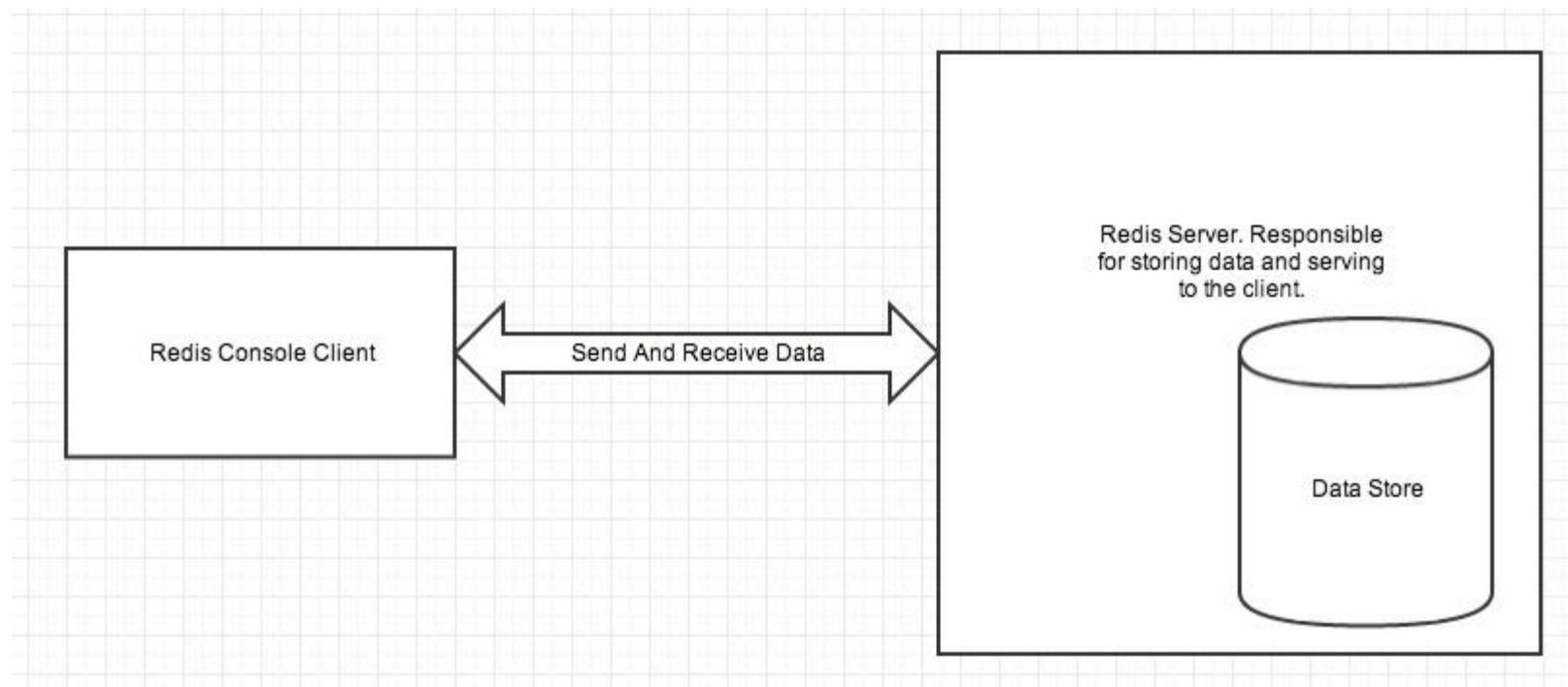
Here name and profession are keys. And we have their respective values on right.

How Redis works?



Redis Single Instance Architecture

Redis architecture contains two main processes: Redis client and Redis Server.



Redis Single Instance Architecture

Redis client and server **can be in the same computer or in two different computers.**

Redis server is responsible for storing data in memory. It handles all kinds of management and forms the major part of architecture. **Redis client** can be Redis console client or any other programming language's Redis API.

As we saw that Redis stores everything in primary memory. **Primary memory is volatile and therefore we will loose all stored data once we restart our Redis server or computer.** **Therefore we need a way for datastore persistence.**

Redis Persistence

There are three different ways to make Redis persistence: RDB, AOF and SAVE command.

RDB Mechanism:

RDB makes a copy of all the data in memory and stores them in secondary storage(permanent storage). This happens in a specified interval. So there is chance that you will loose data that are set after RDB's last snapshot.

Redis Persistence

AOF

AOF logs all the write operations received by the server. Therefore everything is persistence. The problem with using AOF is that it writes to disk for every operation and it is a expensive task and also size of AOF file is large than RDB file.

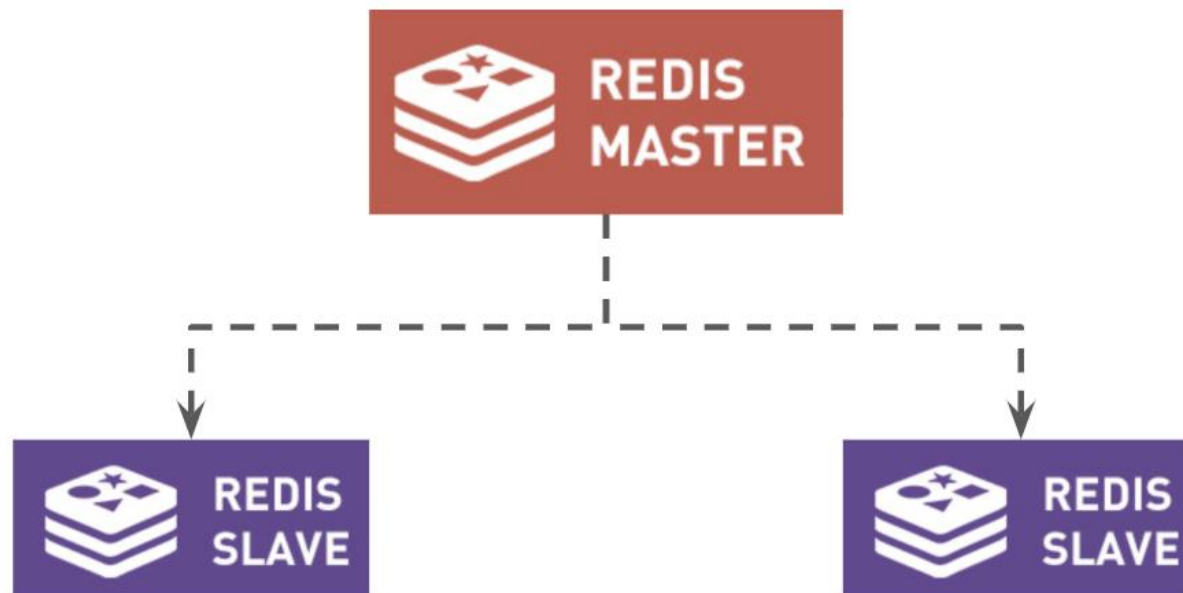
SAVE Command

You can force redis server to create a RDB snapshot anytime using the redis console client SAVE command.

You can use AOF and RDB together to get best persistence result.

Redis Master/Slave

Redis replication is very simple to use and configure master-slave replication that allows slave Redis servers to be exact copies of master servers.



Redis Master/Slave

When using Master-Slave architecture

- There will be only one Master with multiple slaves for replication.
- All write goes to Master, which creates more load on the master node.
- If the Master goes down, the whole architecture is prone to SPOF (Single point of failure).
- M-S architecture does not help in scaling when your user base grows.
- So we need **a process to Monitor Master in case of failure or shutdown, that is Sentinel.**

Redis Sentinel

Redis Sentinel **provides high availability for Redis**. In practical terms this means that using Sentinel you can create a Redis deployment that resists without human intervention certain kinds of failures.

Redis Sentinel also provides other collateral tasks such as

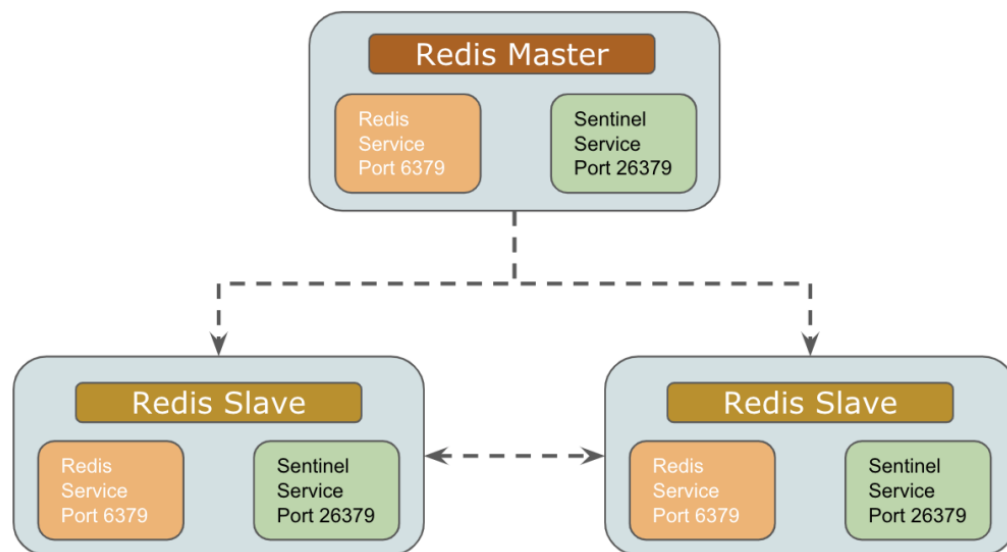
- **Monitoring.** Sentinel constantly checks if your master and replica instances are working as expected.
- **Notification.** Sentinel can notify the system administrator, or other computer programs, via an API, that something is wrong with one of the monitored Redis instances.

Redis Sentinel

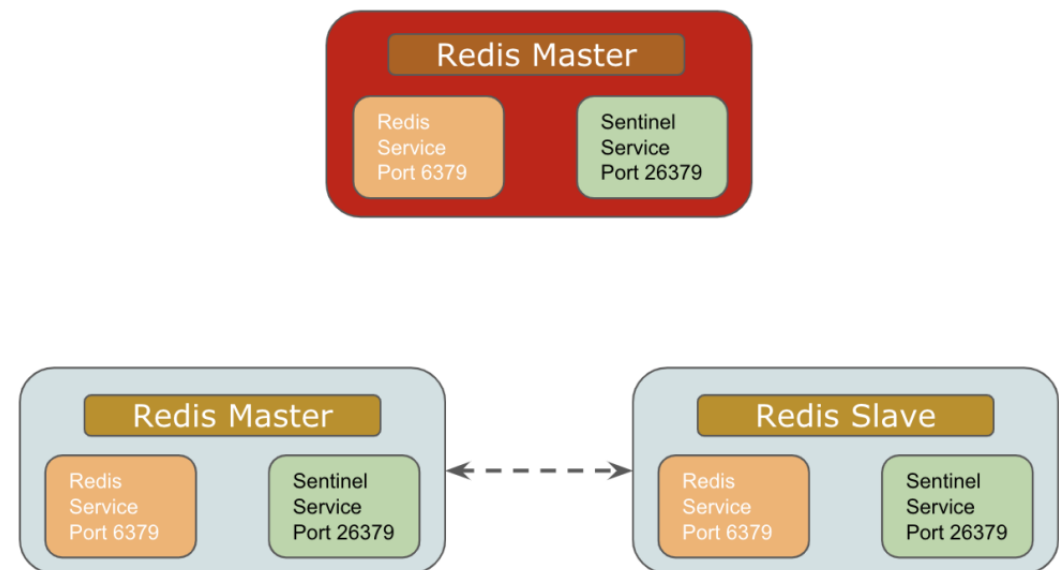
- **Automatic failover.** If a master is not working as expected, Sentinel can start a failover process where a replica is promoted to master, the other additional replicas are reconfigured to use the new master, and the applications using the Redis server are informed about the new address to use when connecting.
- **Configuration provider.** Sentinel acts as a source of authority for clients service discovery: clients connect to Sentinels in order to ask for the address of the current Redis master responsible for a given service. If a failover occurs, Sentinels will report the new address.

Redis Sentinel

Initial Setup



Failover Handling



Redis Cluster

Redis Cluster is an **active-passive cluster implementation that consists of master and slave nodes.**

The cluster uses hash partitioning to split the keyspace into 16,384 key slots, with each master responsible for a subset of those slots.

Each slave replicates a specific master and can be reassigned to replicate another master or be elected to a master node as needed.

Ports Communication

Each node in a **cluster requires two TCP ports**.

One port is used for client connections and communications. This is the port you would configure into client applications or command-line tools.

The second required port is reserved for node-to-node communication that occurs in a binary protocol and allows the nodes to discuss configuration and node availability.

Redis Cluster

Failover

When a master fails or is found to be unreachable by the majority of the cluster as determined by the nodes communication via the gossip port, the remaining masters hold a vote and elect one of the failing masters' slaves to take its place.

Rejoining The Cluster

When the failing master eventually rejoins the cluster, it will join as a slave and begin to replicate another master.

Sharding

Redis sharded data automatically into the servers.

Redis has a concept of the hash slot in order to split data. All the data are divided into slots.

There are 16384 slots. These slots are divided by the number of servers.

If there are 3 servers; A, B and C then

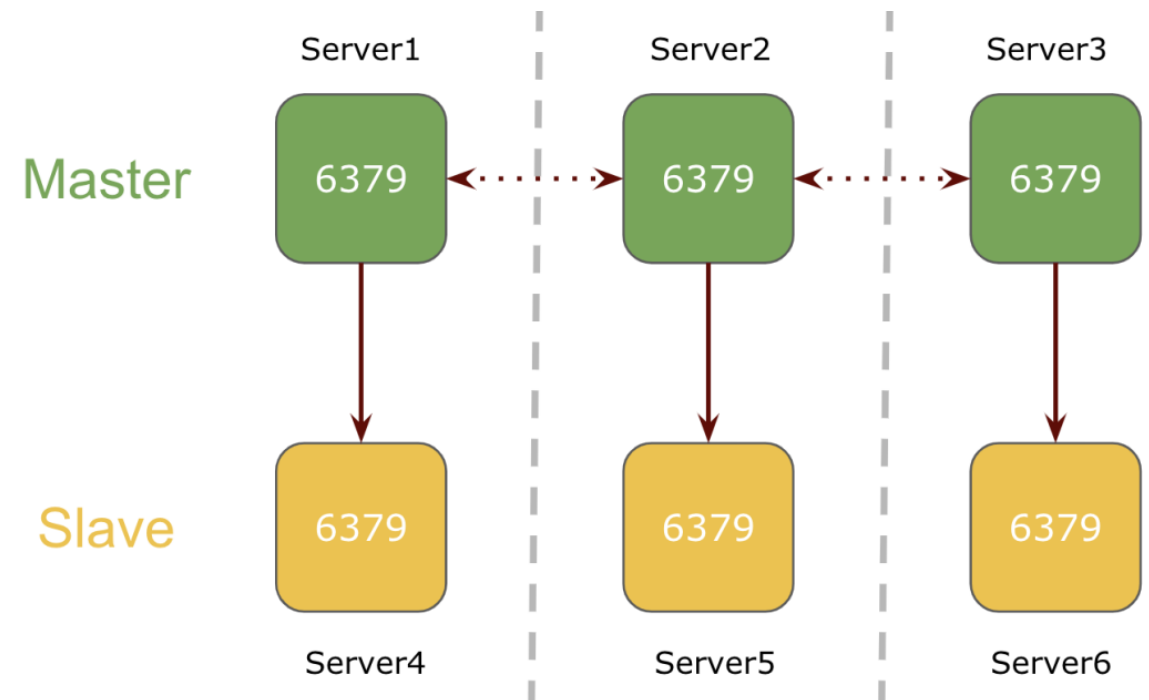
- Server 1 contains hash slots from 0 to 5500.
- Server 2 contains hash slots from 5501 to 11000.
- Server 3 contains hash slots from 11001 to 16383.

Sharding with 6 Node M/S Cluster

In a 6 node cluster mode, 3 nodes will be serving as a master and the 3 nodes will be their respective slave.

Here, Redis service will be running on port 6379 on all servers in the cluster.

Each master server is replicating the keys to its respective Redis slave node assigned during the cluster creation process.

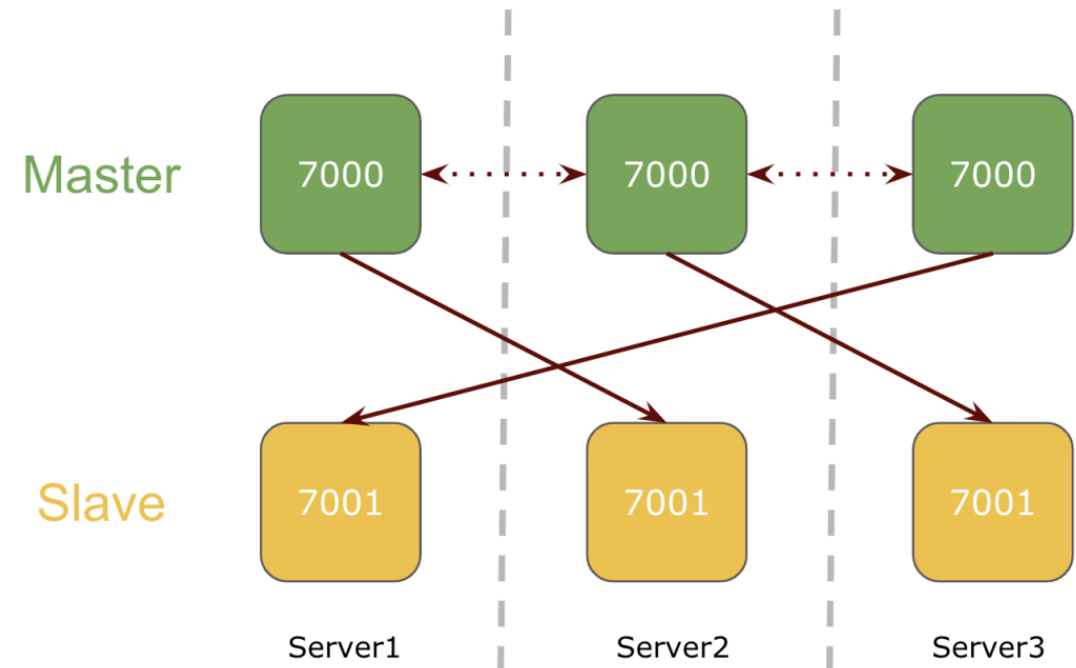


Sharding with 3 Node M/S Cluster

In a 3 node cluster mode, there will be 2 redis services running on each server on different ports.

All 3 nodes will be serving as a master with redis slave on cross nodes.

Here, two redis services will be running on each server on two different ports and each master is replicating the keys to its respective redis slave running on other nodes.

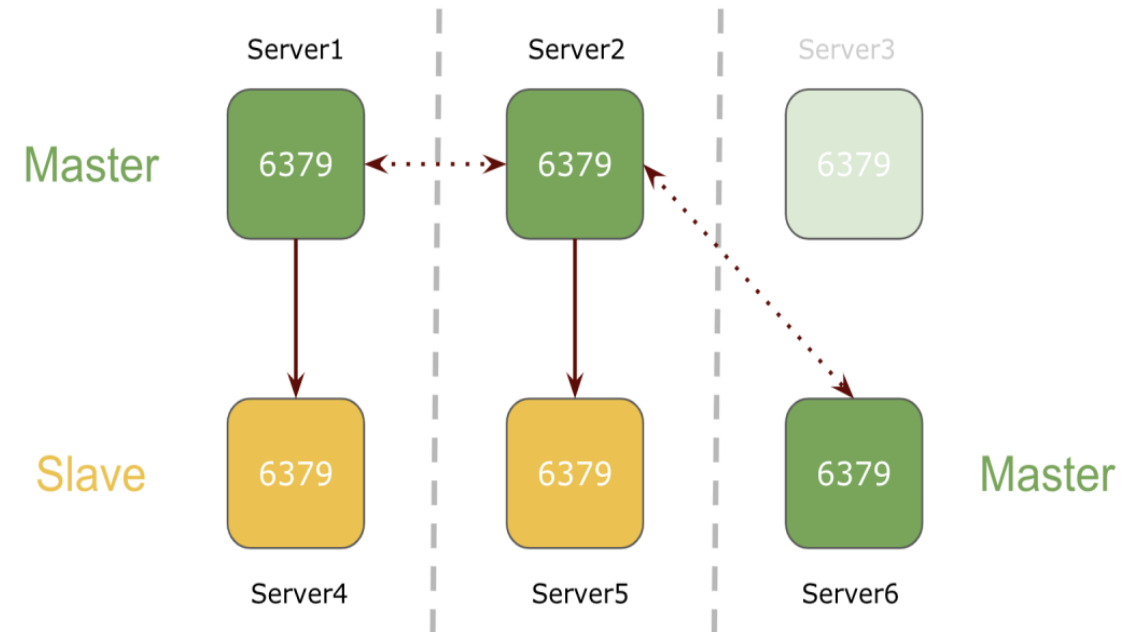


WHAT IF Redis Goes Down

1 node goes down in a 6 node Redis Cluster

If one of the nodes goes down in the Redis 6-node cluster setup, its respective slave will be promoted as the master.

In the above example, master Server3 goes down and its slave Server6 is promoted as the master.

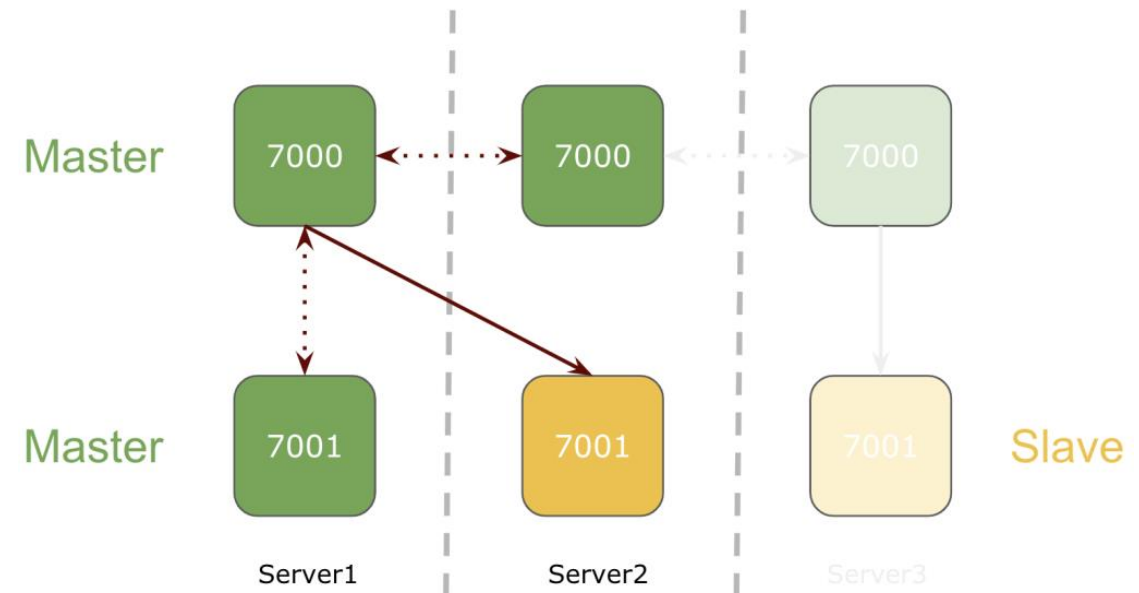


WHAT IF Redis Goes Down

1 node goes down in a 3 node Redis Cluster

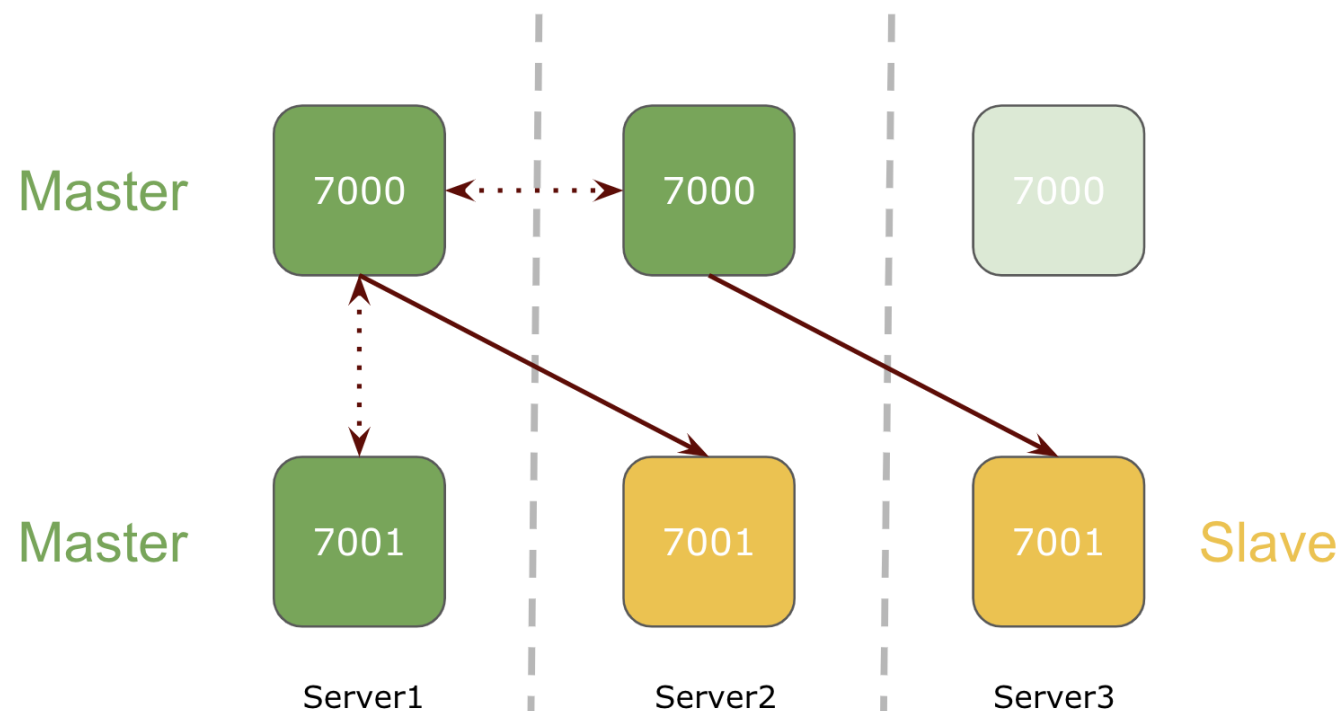
If one of the nodes goes down in the Redis 3-node cluster setup, its respective slave running on the separate node will be promoted to master.

In the above example, Server 3 goes down and slave running on Server1 is promoted to master.



WHAT IF Redis Goes Down

If Redis service goes down on one of the nodes in Redis 3-node cluster setup, its respective slave will be promoted as the master.



Redis over DBMS

Database Management systems store everything in second storage which makes read and write operations very slow. But Redis stores everything in primary memory which is very fast in read and write of data.

Primary memory is limited(much lesser size and expensive than secondary) therefore Redis cannot store large files or binary data. It can only store those small textual information which needs to be accessed, modified and inserted at a very fast rate. If we try to write more data than available memory then we will receive errors.

Redis – Programming Support

You can use Redis from

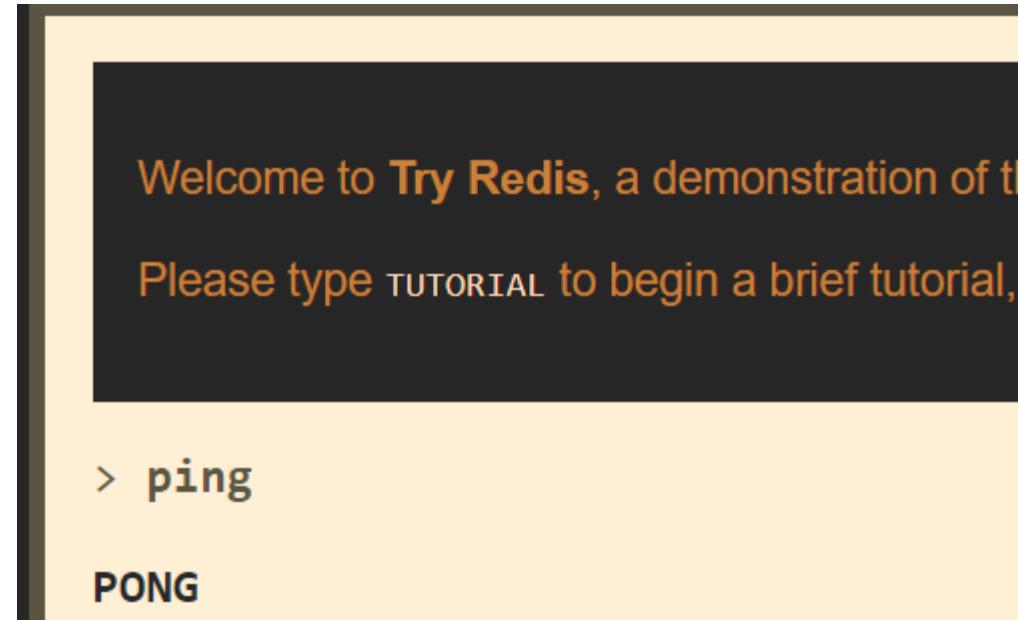
ActionScript	Bash	C	C#	C++	Clojure
Common Lisp	Crystal	D	Dart	Delphi	Elixir
emacs lisp	Erlang	Fancy	gawk	GNU Prolog	Go
Haskell	Haxe	Io	Java	Julia	Lasso
Lua	Matlab	mruby	Nim	Node.js	Objective-C
OCaml	Pascal	Perl	PHP	PL/SQL	Pure Data
Python	R	Racket	Rebol	Ruby	Rust
Scala	Scheme	Smalltalk	Swift	Tcl	VB
VCL	Xojo				

Clients of Redis

- Twitter
- GitHub
- Weibo
- Pinterest
- Snapchat
- StackOverflow
- Flickr
- Digg

Redis installation

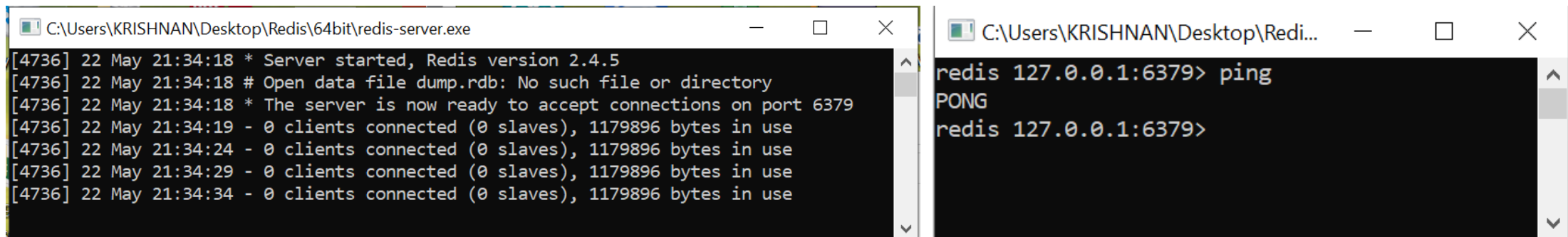
- Linux (Any distro)
- Follow the steps in : <https://redis.io/topics/quickstart>
- For online DB: <http://try.redis.io/>

A screenshot of the Try Redis web interface. It features a dark grey terminal window with orange text. The text reads: "Welcome to Try Redis, a demonstration of the Redis database." followed by "Please type TUTORIAL to begin a brief tutorial,". Below the terminal window, on a light yellow background, the command "> ping" is entered, and the response "PONG" is displayed below it.

```
Welcome to Try Redis, a demonstration of the Redis database.  
Please type TUTORIAL to begin a brief tutorial,  
  
> ping  
  
PONG
```

Redis installation in Windows

- Download Redis from: <https://github.com/dmajkic/redis/downloads>
- Unzip the folder
- Start Redis server (Do not close the terminal)
- Start Redis client



```
C:\Users\KRISHNAN\Desktop\Redis\64bit\redis-server.exe
[4736] 22 May 21:34:18 * Server started, Redis version 2.4.5
[4736] 22 May 21:34:18 # Open data file dump.rdb: No such file or directory
[4736] 22 May 21:34:18 * The server is now ready to accept connections on port 6379
[4736] 22 May 21:34:19 - 0 clients connected (0 slaves), 1179896 bytes in use
[4736] 22 May 21:34:24 - 0 clients connected (0 slaves), 1179896 bytes in use
[4736] 22 May 21:34:29 - 0 clients connected (0 slaves), 1179896 bytes in use
[4736] 22 May 21:34:34 - 0 clients connected (0 slaves), 1179896 bytes in use

C:\Users\KRISHNAN\Desktop\Redis\64bit\redis-cli.exe
redis 127.0.0.1:6379> ping
PONG
redis 127.0.0.1:6379>
```

Summary

This session will give the knowledge about

- Redis introduction and basics
- Redis installation on Windows
- Reference: <https://redis.io/documentation>
- Reference: Redis for Dummies